NLP 243 Deep Learning for Natural Language Processing

Fall 2024

Outline for Today

- NLP basic
- Getting ready for homeworks: knowledge graphs and querying knowledge graphs
- Gradient Descent (and versions)
- Readings:
  - Ch2 of NLP with PyTorch
  - Starting Ch3 of NLP with PyTorch
  - Ch3 of Dive into Deep Learning

Corpus Based Statistical NLP

- Based on counting in large corpora and developing different kinds of models
- This approach only became possible with the Web
  - Google Translate: works because it’s seen tons of examples
  - Spelling Correction: also works because it’s seen tons of examples of words in context
  - Auto-complete
  - Word2Vec
- Started out with simple word and POS based approaches, now NLP challenges are more to do with meaning (i.e., GLUE and SuperGLUE).

Prior knowledge

- Sometimes, an estimation of the probability of an event is affected by what is known (which changes the sample space)

Example:

- A box of 10 candies
- **5 red and sweet**, **3 green and sour**, **2 green and sweet**
- You close your eyes and randomly pick a candy, what is the probability that it is sweet?
  - 7/10 = 0.7
- After you pick the candy, you open your eyes and see it is green, now what is the probability that it is sweet?
  - 2/5 = 0.4

Part-of-speech tagging example

- One of the classic tasks in statistical NLP
- Assign a label indicating the grammatical category to every word in a corpus of running text.
- Statistical POS taggers are first trained on data that has been previously annotated. Yields a **language model**.

- Language models vary based on the n-gram window (size of the sequence):
- Unigrams: probability based on tokens (a lexicon)

e.g. input = the_DET tall_ADJ man_NN

model represents the probability that the word *man* is a noun (it could also be a verb)

- Bigrams: probabilities across a span of 2 words

model represents the probability that a DET is followed by an adjective, adjective is followed by a noun, etc.

- Also: Trigrams, 4-grams, etc.

POS tagging

- Suppose we've trained a tagger on the annotated data. It has:
  - a lexicon of unigrams:
    - P(the=DET), P(man=NN), etc
  - a bigram model
    - P(DET is followed by ADJ), etc
  - Assume we've trained it on a large input sample.
  - We now feed it a new phrase:
    - *the audacious alien*
- Our tagger knows that the word *the* is a DET, but it's never seen the other words.
- It can:
  - Make a wild guess (not very useful!)
  - Alternative: **estimate the probability** that *the* is followed by an ADJ, and that an ADJ is followed by a NOUN

Prior knowledge revisited

- Given that I know that *the* is DET, what's the probability that the following word *audacious* is ADJ?
  - This is **very different** from asking what's the probability that *audacious* is ADJ out of context.
  - We have **prior knowledge** that DET has occurred. This can significantly change the estimate of the probability that *audacious* is ADJ.
- We therefore distinguish:
  - **Prior probability**: "Naïve" estimate based on long-run frequency
    - The unconditional probability that is assigned before any relevant evidence is taken into account.
  - **Posterior probability**: probability estimate based on prior knowledge after some observation
    - The **conditional probability** that is assigned after the relevant evidence or background is taken into account.

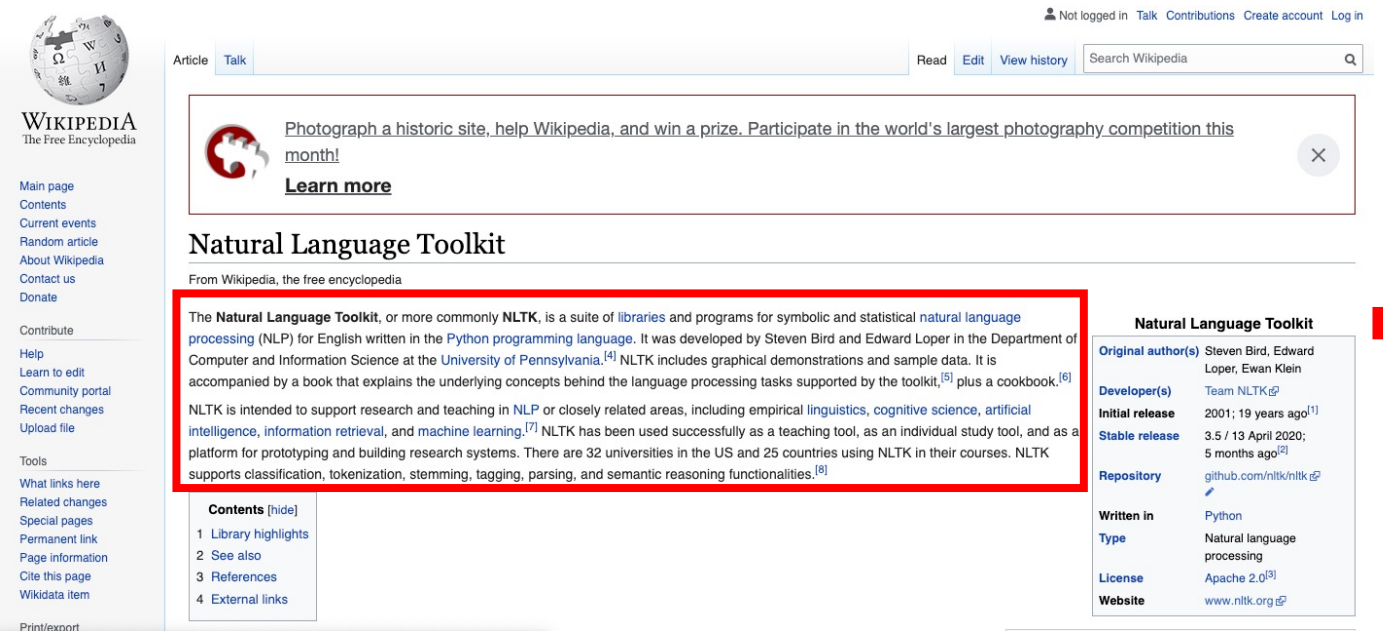
Conditional Probability

- In our example, we were estimating:
  - P(ADJ|DET) = probability of ADJ given DET
  - P(NN|ADJ) = probability of NN given ADJ
  - …

- In general:
  - The conditional probability P(A|B) is the probability that A occurs, given that we know that B has occurred

NLP Basic Terminology

- **Corpus** (plural, corpora): Large body of natural language data, (usually raw) text.
  - Examples include Brown corpus, Google Books corpus, Wikipedia, etc.



Break into sentences and words

Natural Language Toolkit (NLTK)

https://www.nltk.org

- **Sentence Segmentation:** breaking text down into its sentences.
- **Tokenization:** breaking text down into its tokens (words, numbers, punctuation).

- Sentence:
  - Something ending with a ., ?, ! (and sometimes also :)
- Most ambiguities are from abbreviations, for example:
  - The house located on Main St. was owned by the son.
  - The new U.N. decisions about 3.5 years of …
- Nested sentences:
  - “You reminded me,” she remarked, “of your mother.”
    - Note the .”

Natural Language Toolkit (NLTK)

https://www.nltk.org

- **Sentence Segmentation:** breaking text down into its sentences.
- **Tokenization:** breaking text down into its tokens (words, numbers, punctuation).

- Tokens:
  - an indivisible sequence of characters
  - carry elementary meaning
  - reusable in different contexts

- More challenging than expected:
  - “He’s” versus “he is”

Natural Language Toolkit (NLTK)

Natural Language Toolkit (NLTK)

https://www.nltk.org

- **Sentence Segmentation:** breaking text down into its sentences.
- **Tokenization:** breaking text down into its tokens (words, numbers, punctuation).

```
[1] import nltk
    # nltk.download('punkt') # you need this when calling for the first time.
```

```
[2] nltk.sent_tokenize("She had a brown cat. It purred when someone called her.")
```

```
['She had a brown cat.', 'It purred when someone called her.']
```

```
[3] nltk.word_tokenize("Show me movies directed by Woody Allen recently.")
```

```
['Show', 'me', 'movies', 'directed', 'by', 'Woody', 'Allen', 'recently', '.']
```

Punkt Sentence Segmenter

Natural Language Toolkit (NLTK)

https://www.nltk.org

- NLTK uses the *Punkt* sentence segmenter (Kiss and Strunk, 2006)
- Assumption: a large number of ambiguities in the determination of sentence boundaries can be eliminated once abbreviations have been identified.
- Uses 3 simple rules to identify them
  - look for very tight collocation consisting of a truncated word and a final period
  - Abbreviations are usually short
  - Abbreviations can contain internal periods
- Language independent.

Sentence Segmentation as Classification

- D. Gillick. Sentence boundary detection and the problem with the U.S., NAACL, 2009.
- Fully statistical system, publicly available at: https://code.google.com/archive/p/splitta/
- For every word boundary with a candidate token:

  <preceding word>. <following word>

  such as "St. was", a set of features are computed.
- For example: len(<preceding word>)
- Uses Support Vector Machine classifiers to train a model that would estimate if the boundary is a sentence boundary or not, given the associated features.

NLP Basic Terminology (cont.)

- Ngrams: fixed-length (*n*) consecutive token sequences occurring in the text.
  - unigrams: show, me, movies, directed, …
  - Bigrams: show me, me movies, movies directed, …
  - Trigrams: show me movies, me movies directed, …

- Lemmas: *Lemmas* are root forms of words.
  - Verb *fly*. It can be *inflected* into many different words—*flow*, *flew*, *flies*, *flown*, *flowing*—
  - *fly* is the lemma

https://books.google.com/ngrams

Counting in NLTK

- Word frequencies, etc.

```
[7] nltk.download('book')
    from nltk.book import *
```

Corpora in NLTK!

```
[8] text1
```

```
<Text: Moby Dick by Herman Melville 1851>
```

```
text1.concordance('child')
```

```
Displaying 18 of 18 matches:
e try to explain them . When I was a child , I well remember a somewhat similar
th loud lament the parents saw their child borne out of sight over the wide wat
y that sweet girl that old man has a child : hold ye then there can be any utte
 memories of his young Cape wife and child , tend to bend him still more from t
tender than the skin of a new - born child . But no more of this . Assuming the
 he would have been deified by their child - magian thoughts . They deified the
shery , that they are accounted mere child ' s play . Some one strips off a fro
hings tapers off at last into simple child ' s play . CHAPTER 104 The Fossil Wh
AT is all wrinkles ! But , away with child ' s play ; no more gaffs and pikes t
ire thou madest me , and like a true child of fire , I breathe it back to thee
buck may survive to hug his wife and child again .-- Oh Mary ! Mary !-- boy ! b
```

Counting in NLTK (cont.)

```
words = [word.lower() for word in text1]
print(len(words))
print(words)
```

```
260819
['[', 'moby', 'dick', 'by', 'herman', 'melville', '1851', ']', 'etymology', '.', '(', '
```

```
fdist = nltk.FreqDist(words)
fdist.N()
```

```
260819
```

```
[16] fdist.most_common(10)
```

```
[(',', 18713),
 ('the', 14431),
 ('.', 6862),
 ('of', 6609),
 ('and', 6430),
```

Part-of-speech Tagging in NLTK

- Sometimes lexical items are too specific:
  - Don't generalize well
  - Sparse data problems
- Grouping words into a small set of word classes or lexical categories can be useful for many applications
- Part-of-speech is a common method of categorizing words
  - Grounded in linguistic theory

```
nltk.pos_tag(text1)
```

```
('visited', 'VBD'),
('this', 'DT'),
('country', 'NN'),
('also', 'RB'),
('with', 'IN'),
('a', 'DT'),
('view', 'NN'),
('of', 'IN'),
('catching', 'VBG'),
```

```
nltk.help.upenn_tagset("RB")
```

```
RB: adverb
    occasionally unabatingly maddeningly adventurously professedly
    stirringly prominently technologically magisterially predominately
    swiftly fiscally pitilessly ...
```

NLP Basic Terminology (cont.)

spaCy
https://spacy.io/

- Using Spacy

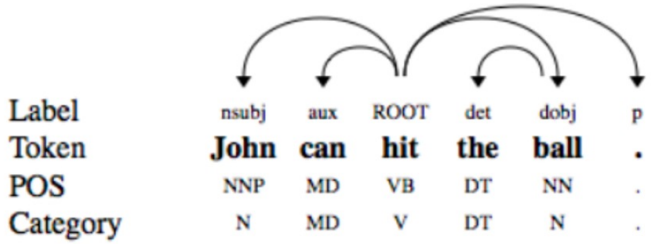
```
[38] import spacy

     nlp = spacy.load("en")

     text = "Show me movies directed by Woody Allen recently."
     doc = nlp(text)
     for token in doc:
       print("%s %s %s %s %s %s" %
             (token, token.lemma_, token.pos_, token.dep_, token.head, token.ent_type_))
```

```
Show show VERB ROOT Show
me -PRON- PRON dative Show
movies movie NOUN dobj Show
directed direct VERB acl movies
by by ADP agent directed
Woody Woody PROPN compound Allen PERSON
Allen Allen PROPN pobj by PERSON
recently recently ADV advmod directed
. . PUNCT punct Show
```

POS tags, dependency parse tree, named entities



NLP Basic Terminology (cont.)

- Chunking
- *"Show me movies directed by Woody Allen recently."*

```
[31] for chunk in doc.noun_chunks:
       print(chunk)
```

```
me
movies
Woody Allen
```

- Chunking
  - breaking down a difficult text into more manageable pieces
  - i.e., noun phrases

NLP Basic Terminology (cont.)

- Named entity tagging

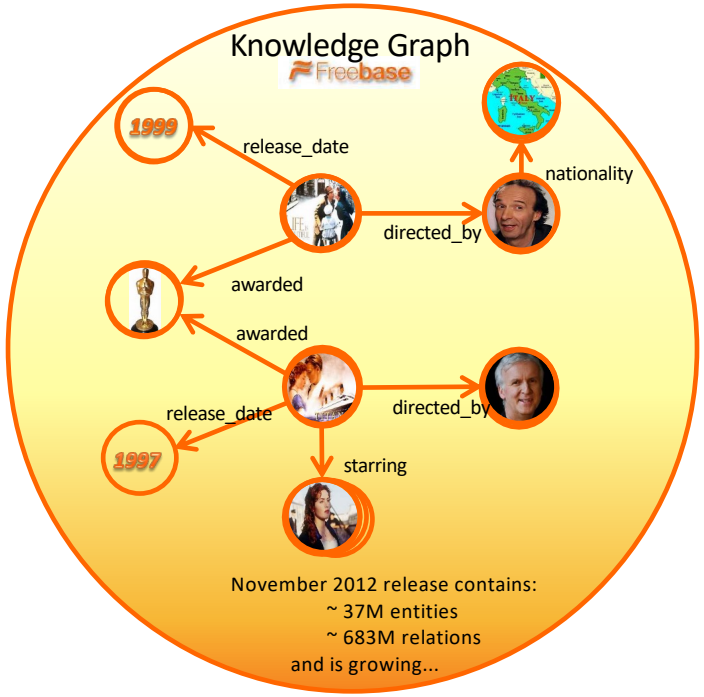
```
[40] text = "Heywood Allen better known as Woody Allen was born in 1935."
     doc = nlp(text)
     for ent in doc.ents:
       print("%s %s %s %s" % (ent.text, ent.start_char, ent.end_char, ent.label_))
```

```
Heywood Allen 0 13 PERSON
Woody Allen 30 41 PERSON
1935 54 58 DATE
```

Named entities:
Person, location, organization, date, number, etc.

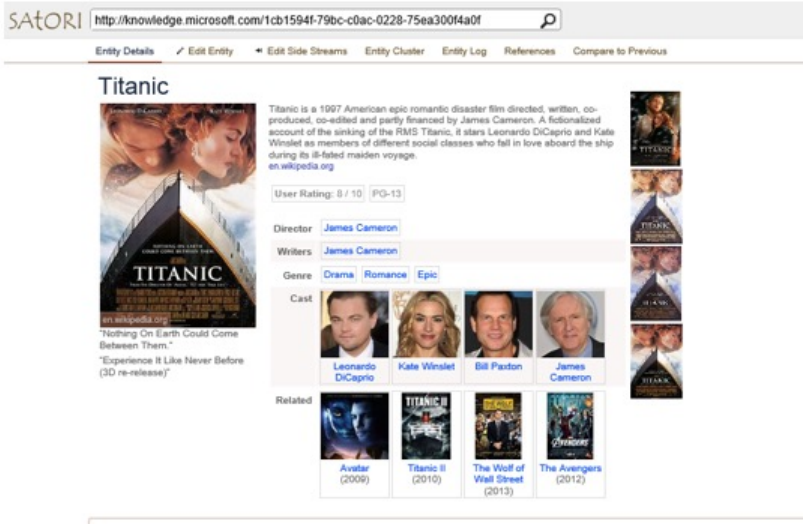
Representing Relational Information

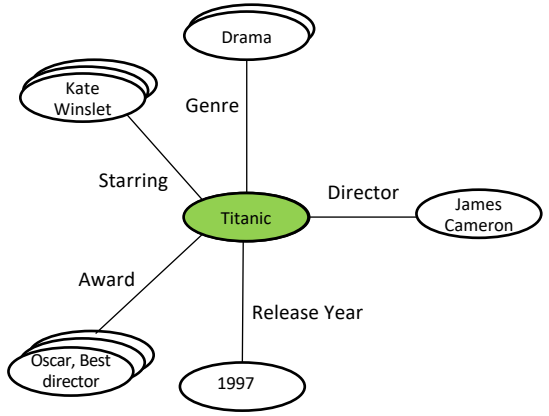
- **Knowledge Graphs:** Graphs of strongly typed and uniquely identified **entities** (nodes) and facts/literals connected by **relations** (edges)
- Focus on entities and their relations, represented as subject, predicate, object triples:
  - <Avatar, Director, James Cameron>
  - <Barrack Obama, Spouse, Michelle Obama>
  - Can issue queries in graph query languages, such as SPARQL.



Knowledge Graphs

- **Examples**
  - Satori, Google KG, Facebook Open Graph, Freebase, Wikidata…
- **How large?**
  - > 500M entities, >1.5B relations
- **How broad is the knowledge?**
  - *Wikipedia-breadth*: "American Football" ← → "Zoos"



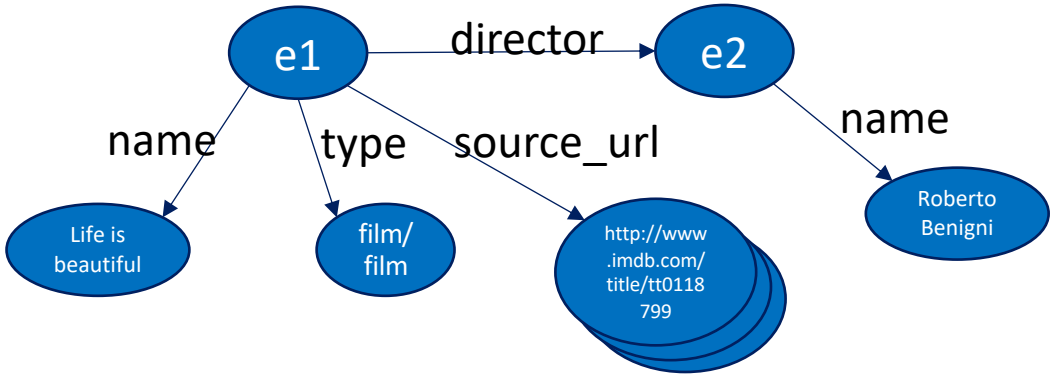


Knowledge Graphs

- RDF: Resource description framework, W3C standard for encoding knowledge.
- Graph structures to represent and store knowledge
- Example “triples”:

Subject, Predicate, Object

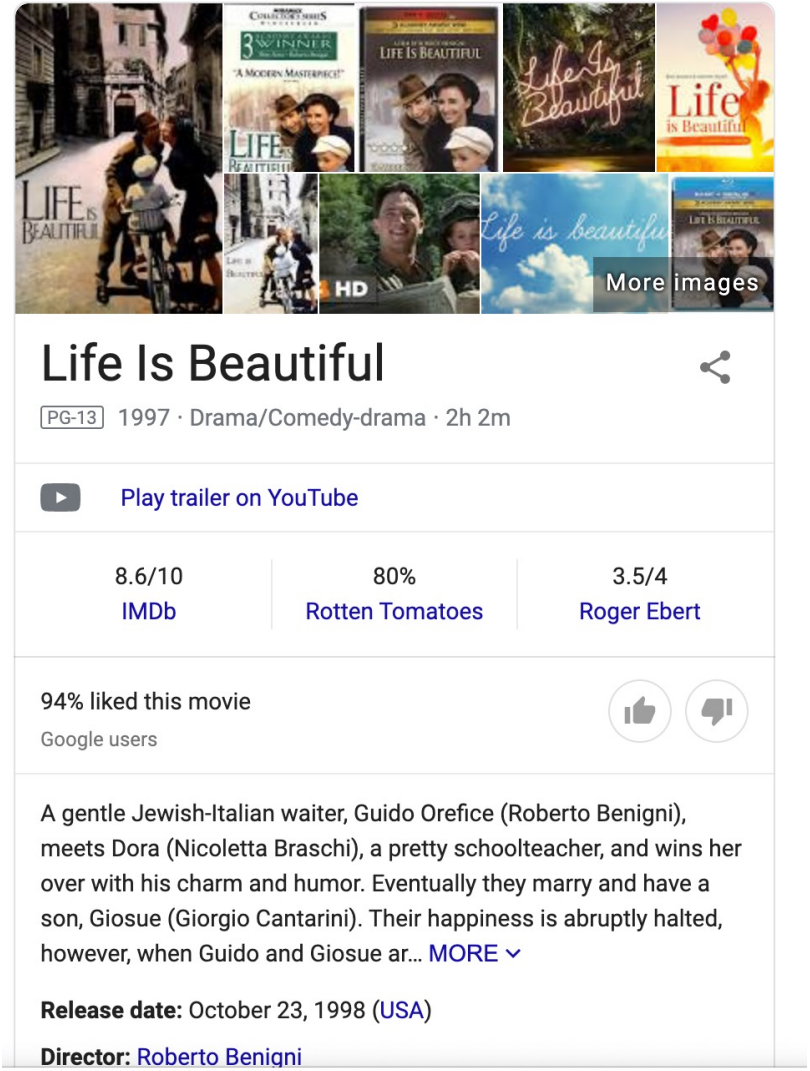
e1 name “Life is beautiful”.
e1 type film/film.
e1 source_url http://www.imdb.com/title/tt0118799.
e1 director e2.
e2 name “Roberto Benigni”.



Why Knowledge Graphs?

- Efficient way to represent knowledge!
- Useful for many NLP problems, as well as web search!
- For example, one can use entities in a knowledge graph to tag entities and their relations.

Who directed Life is Beautiful?

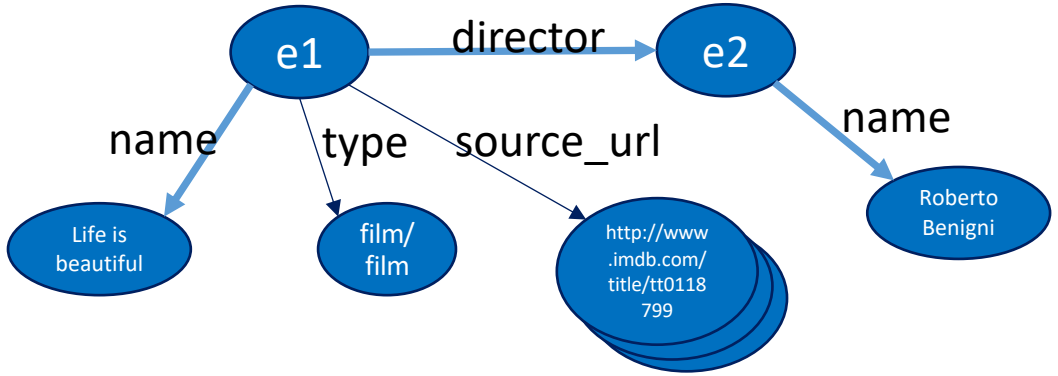


Query Languages for Graphs: SPARQL

- A query language for querying knowledge graphs.
- Each query is a sub-graph.
- Example natural language query:

  Who directed the movie Life is Beautiful?

```
SELECT ?dname WHERE {
    ?movie name "Life is beautiful".
    ?movie director ?director.
    ?director name ?dname.
}
```

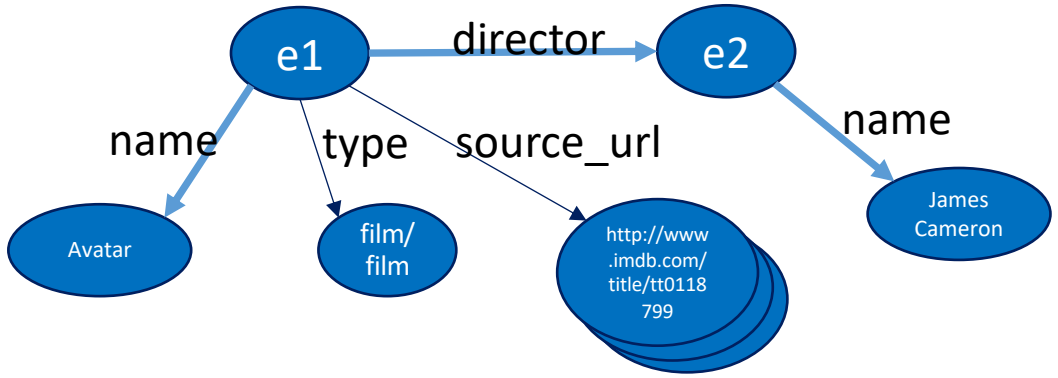


SPARQL

- Querying graph patterns along with their conjunctions and disjunctions.
- Also supports aggregation, sorting, negation, etc.
- Example natural language query:

  How many movies did James Cameron direct?

```
SELECT COUNT(?movie) WHERE {
    ?movie director ?director.
    ?director name "James Cameron".
}
```



Relation Detection

- Aims to detect all relations used in a SPARQL query corresponding to a natural language query.
- The homework will only focus on core relations (i.e., not name relations).
- Text classification task.
- Other similar tasks are:
  - Topic classification
  - Domain/Intent classification
- Multi-class, multi-label classification:
  - There are multiple relation types
  - Each query may be associated with zero, one or more relations!

Relation Detection - Example

NL Query: what movies has tom cruise been in recently

```
SELECT ?movie WHERE {
    ?movie <http://rdf.freebase.com/ns/type.object.type> <http://rdf.freebase.com/ns/film.film>.
    ?movie <http://rdf.freebase.com/ns/film.film.starring> ?cast.
    ?cast <http://rdf.freebase.com/ns/film.performance.actor> ?actor.
    ?actor <http://rdf.freebase.com/ns/type.object.name> "Tom Cruise"@en.
    ?movie <http://rdf.freebase.com/ns/film.film.initial_release_date> ?date.
} ORDER BY DESC(?date)
```

- Core relations: movie.starring.actor
  movie.initial_release_date

Homework

- We will give you a training set of example NL queries, paired with relations in them.

  NL Query: what movies has tom cruise been in recently

  Relations: movie.starring.actor movie.initial_release_date

- We will also give you a test set of NL queries

  NL Query: I am looking for some family friendly movies

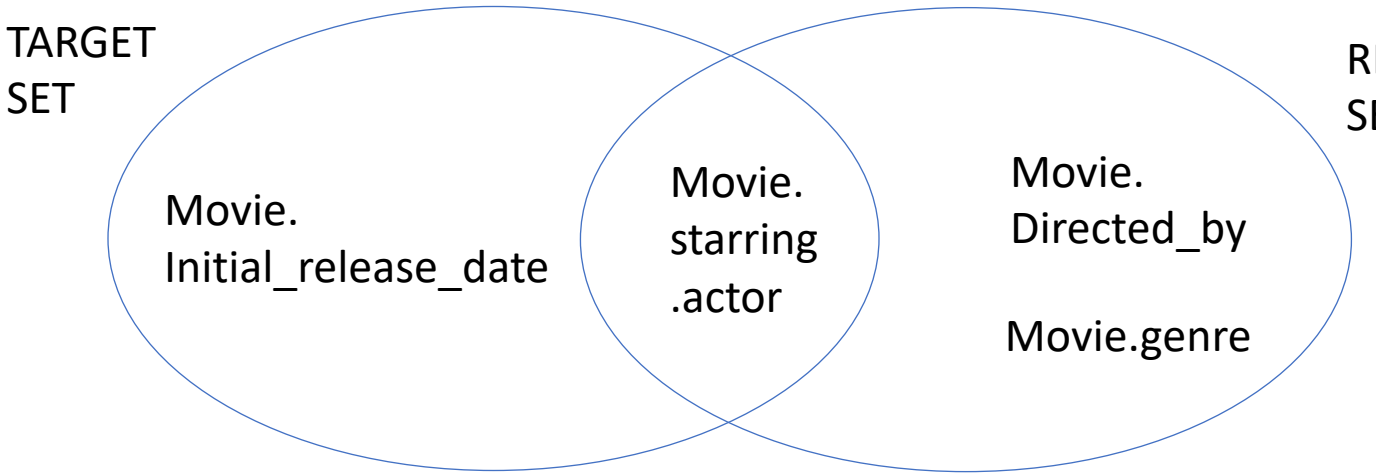
- You will estimate and submit the set of relations in each of these examples.

  Relations: movie.mpaa_rating

Evaluation – F1-measure

- Harmonic mean of recall and precision
- For example, for the example utterance:

  What movies has Tom Cruise has been in recently
- If the classifier returns: movie.starring.actor, movie.genre, and movie.directed_by



True Positives = {movie.starring.actor}

False Positives = {movie.directed_by, movie_genre}

False Negatives = {movie.initial_release_date}

Evaluation – F1-measure (cont.)

- In NLP we are always dealing with these kinds of errors.
- Reducing the error rate for an application often involves two antagonistic efforts:
  - Increasing precision (minimizing false positives)
  - Increasing recall (minimizing false negatives)

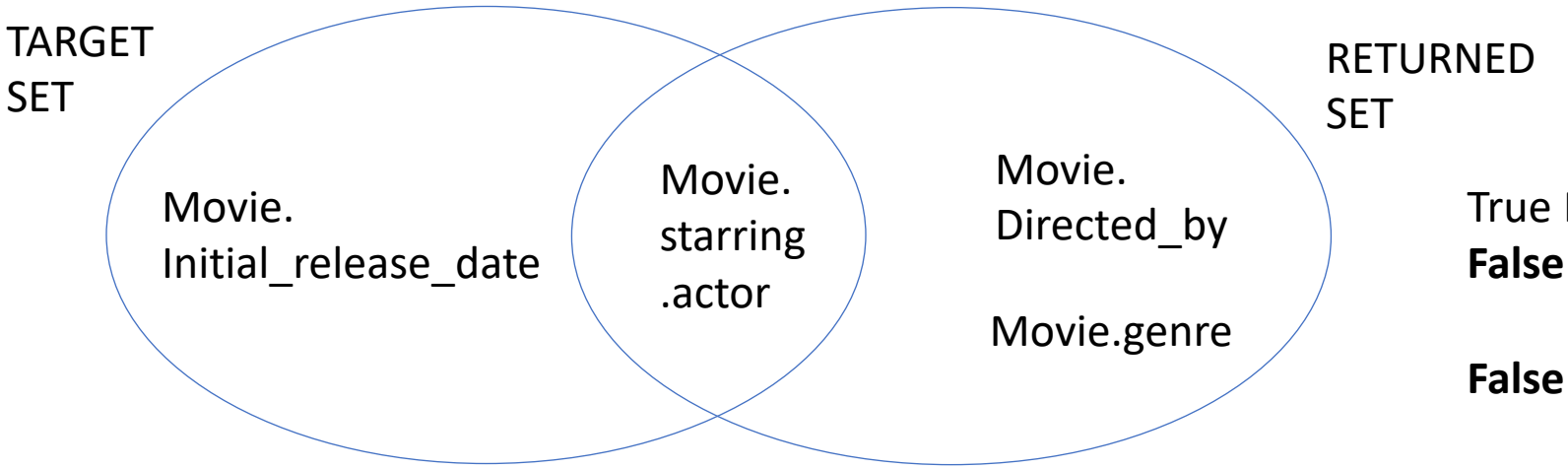
  - Precision = TP / TP + FP
  - Recall = TP / TP + FN
  - F1 = (2 X Precision X Recall) / (Precision + Recall)

Evaluation – F1-measure (cont.)

Recall = ½ = 0.50

Precision = 1/3 = 0.33

F1 = 0.40



True Positives = {movie.starring.actor}

False Positives = {movie.directed_by, movie_genre}

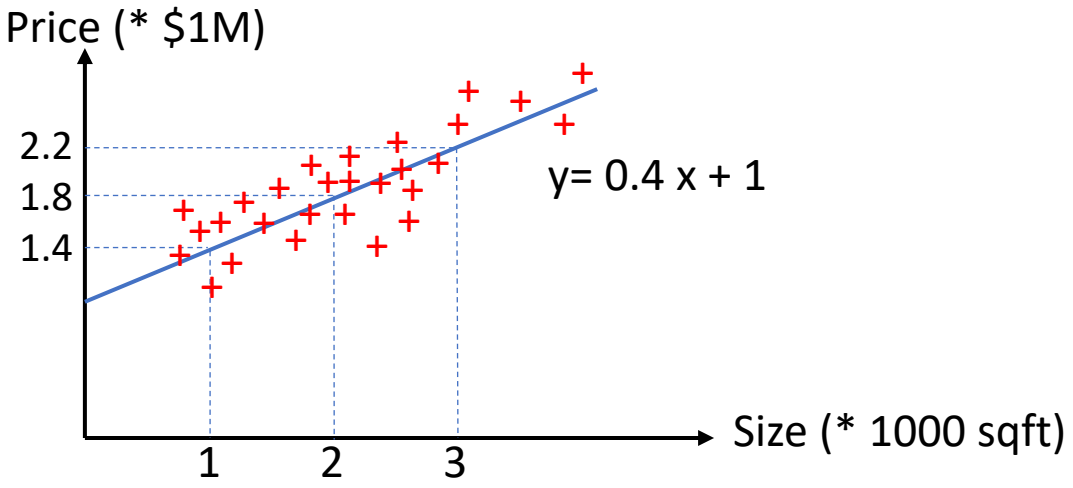
False Negatives = {movie.initial_release_date}

Linear Regression

- **Regression**: modeling the relationship between data points **x** and corresponding real-valued targets y
  - Example: Predicting prices of homes
  - Label: Price, $y^{(i)}$
  - Features: Size, Age
    - $x^{(i)}=[x^{(i)}_1, x^{(i)}_2]$

2019 Sales:

Size (sqft)	Price
2100	$1.90M
1600	$1.56M
3200	$2.40M
…	…



Linear Regression

- Assumption:
  - the relationship between the *features* $\mathbf{x}$ and targets y is linear,
  - i.e., y can be expressed as a weighted sum of the inputs $\mathbf{x}$, give or take some noise on the observations

$$\text{price} = w_1 . x_1 + w_2 . x_1 + b$$

Bias
(offset or intercept)

weights

Linear Regression

- Goal: choose the weights w and bias b to best fit the true values of y observed in the data
- When our inputs consist of d features, we express our prediction $\hat{y}$ as:

$$\hat{y} = w_1 \cdot x_1 + \ldots + w_d \cdot x_d + b$$

- Collecting all features into a vector $\mathbf{x}$ and all weights into a vector $\mathbf{w}$, our model can be expressed compactly using a dot product:

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

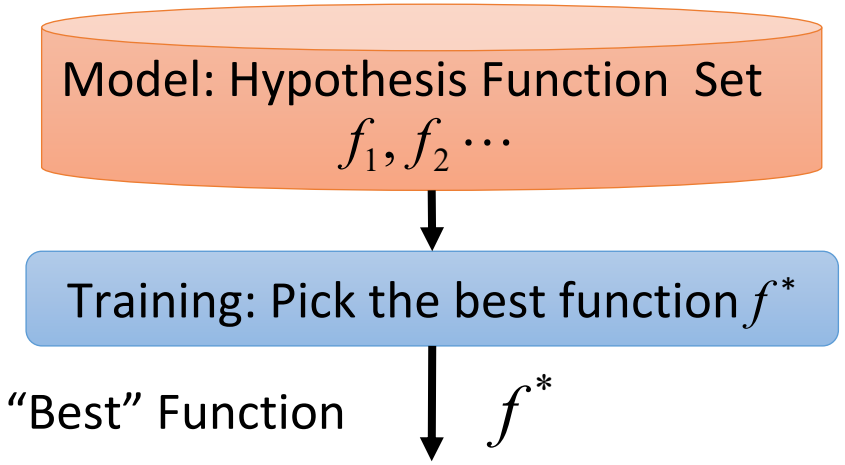
- $\mathbf{x}$**:** feature vector for a single data point
- $\mathbf{X}$**:** a collection of data points
- $\hat{\mathbf{y}}$: vector of estimated predictions

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

Training Procedure: Searching for the best parameters

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

Parameters: elements of **w and** b



- Q1. What is the model? (function hypothesis set)
- Q2. What does a "good" function mean?
- Q3. How do we find the "best" function?

Loss Function

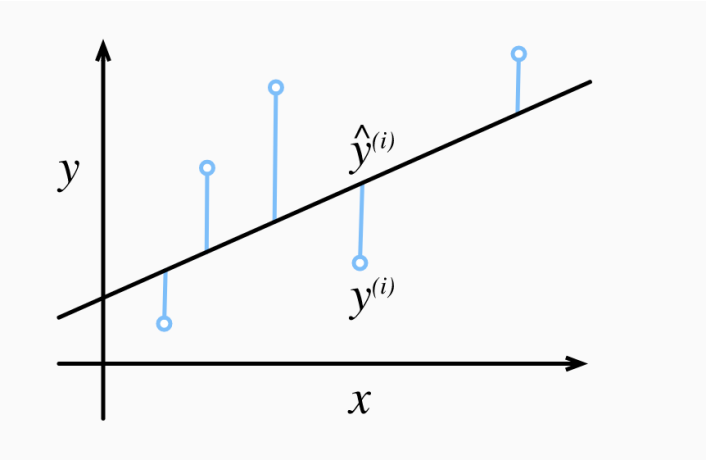
- Measure of *fitness*, quantifies the distance between $\mathbf{y}$ **and** $\hat{\mathbf{y}}$.
- Sum of squared errors:

$$l^{(i)}(\mathbf{w},b) = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2$$

- To measure quality over the entire training set, we average over all K training examples:

$$L(\mathbf{w},b) = \frac{1}{K}\sum_{i=1}^{K} l^{(i)}(\mathbf{w},b)$$

$$= \frac{1}{K}\sum_{i=1}^{K} \frac{1}{2}\left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}\right)^2$$



Residual = truth - predicted

Training Procedure (cont.)

- Training the model ≈ search for parameters ($\mathbf{w}^*, b^*$) that minimize the total loss across all training samples:

$$\mathbf{w}^*, b^* = \text{argmin}_{\mathbf{w},b}\, L(\mathbf{w}, b)$$

Simplification in Notation

- $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$
- Append 1 to features and bias to the weights: **X -> [X 1] and w ->** $\begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$

$$\begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \dots & & & \\ x_1^{(K)} & x_2^{(K)} & \dots & x_n^{(K)} \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_n \end{pmatrix} + b = \begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} & 1 \\ x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} & 1 \\ \dots & & & & \\ x_1^{(K)} & x_2^{(K)} & \dots & x_n^{(K)} & 1 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_n \\ b \end{pmatrix} = \mathbf{X}\,\mathbf{w}$$

Solution for Linear Regression

$$L(\mathbf{X}, y, \mathbf{w}) = \frac{1}{K} \, || \, y - \mathbf{X}.\mathbf{w} \, ||^{2}$$

$$\frac{d}{d\mathbf{w}} L(\mathbf{X}, y, \mathbf{w}) = \frac{2}{K} (y - \mathbf{X}.\mathbf{w})^{\mathsf{T}} \mathbf{X}$$

Loss is convex, hence the optimum solution is at: $\frac{d}{d\mathbf{w}} L(\mathbf{X}, y, \mathbf{w}) = 0$

Therefore,

$$\frac{2}{K} (y - \mathbf{X}.\mathbf{w}^{*})^{\mathsf{T}} \mathbf{X} = 0 \qquad \text{and} \qquad \mathbf{w}^{*} = (\mathbf{X}^{\mathsf{T}} \mathbf{X})^{-1} \mathbf{X} \, y$$

Outline for Today

- Knowledge Graphs, homework 1 review
- Linear Regression
- Gradient Descent (and versions)
- Linear Neural Networks for Regression
- Practical Tips
- Linear Neural Networks for Classification

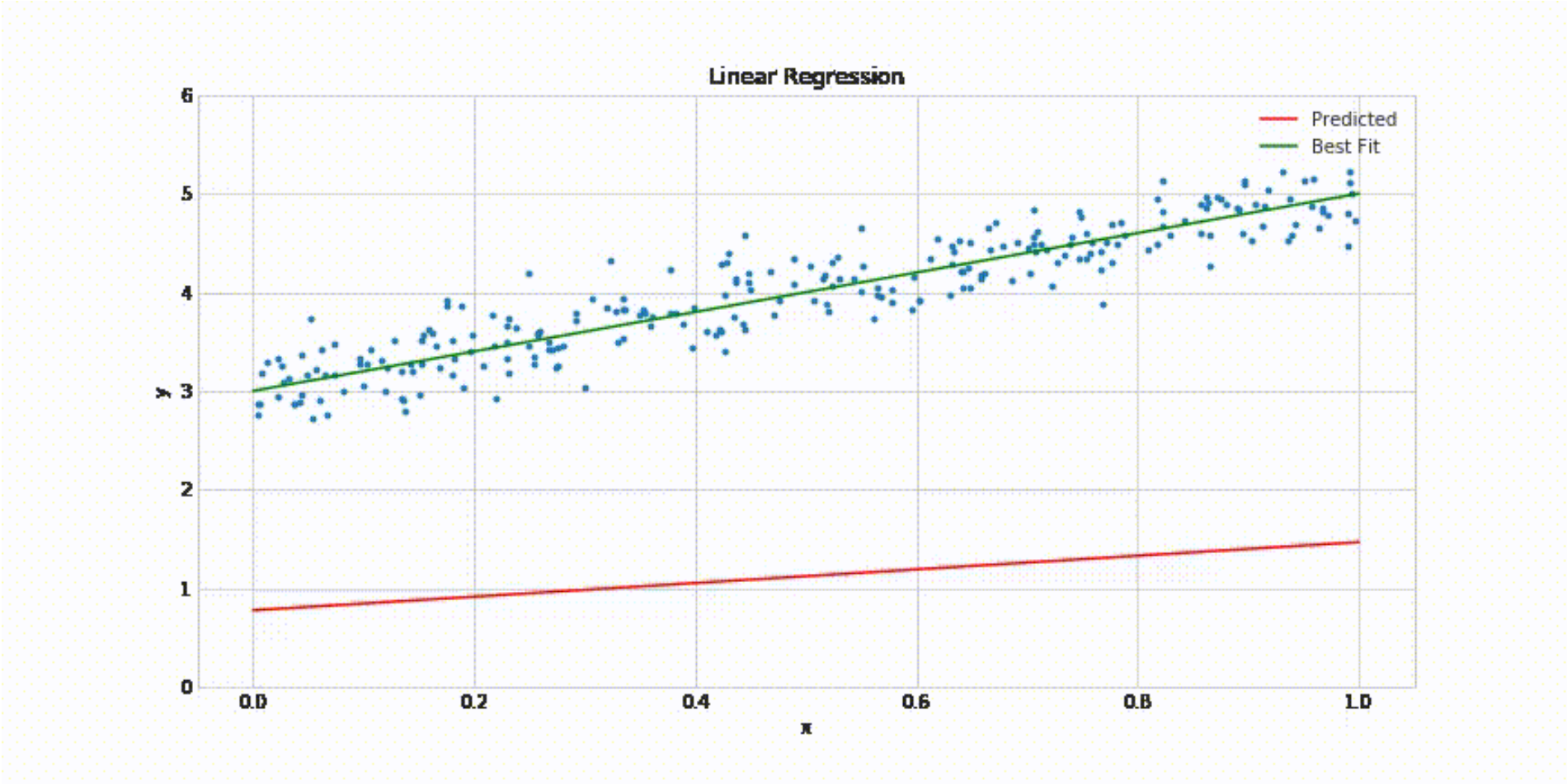
- Readings:
  - Ch2 of NLP with PyTorch
  - Starting Ch3 of NLP with PyTorch
  - Ch3 of Dive into Deep Learning

Gradient Descent

- Even when we cannot solve the models analytically, we can still train models effectively by **iteratively reducing the error by updating the parameters in the direction that incrementally lowers the loss function:**
  - Derivative of the true loss (i.e., average of the losses computed over all examples in the training data).
  - Expressing the updates mathematically:

$$(\mathbf{w},b) \leftarrow (\mathbf{w},b) - \frac{\eta}{|K|} \sum_{i \in K} \partial_{(\mathbf{w},b)} L^{(i)}(\mathbf{w},b)$$

Case Study: $f(x) = mx + b$



Gradient Descent (cont.)

- Steps of the algorithm:
  1. initialize the values of the model parameters, typically at random
  2. iteratively update the parameters in the direction of the negative gradient.

Notation:

θ: model parameters ($\mathbf{w}$,b)

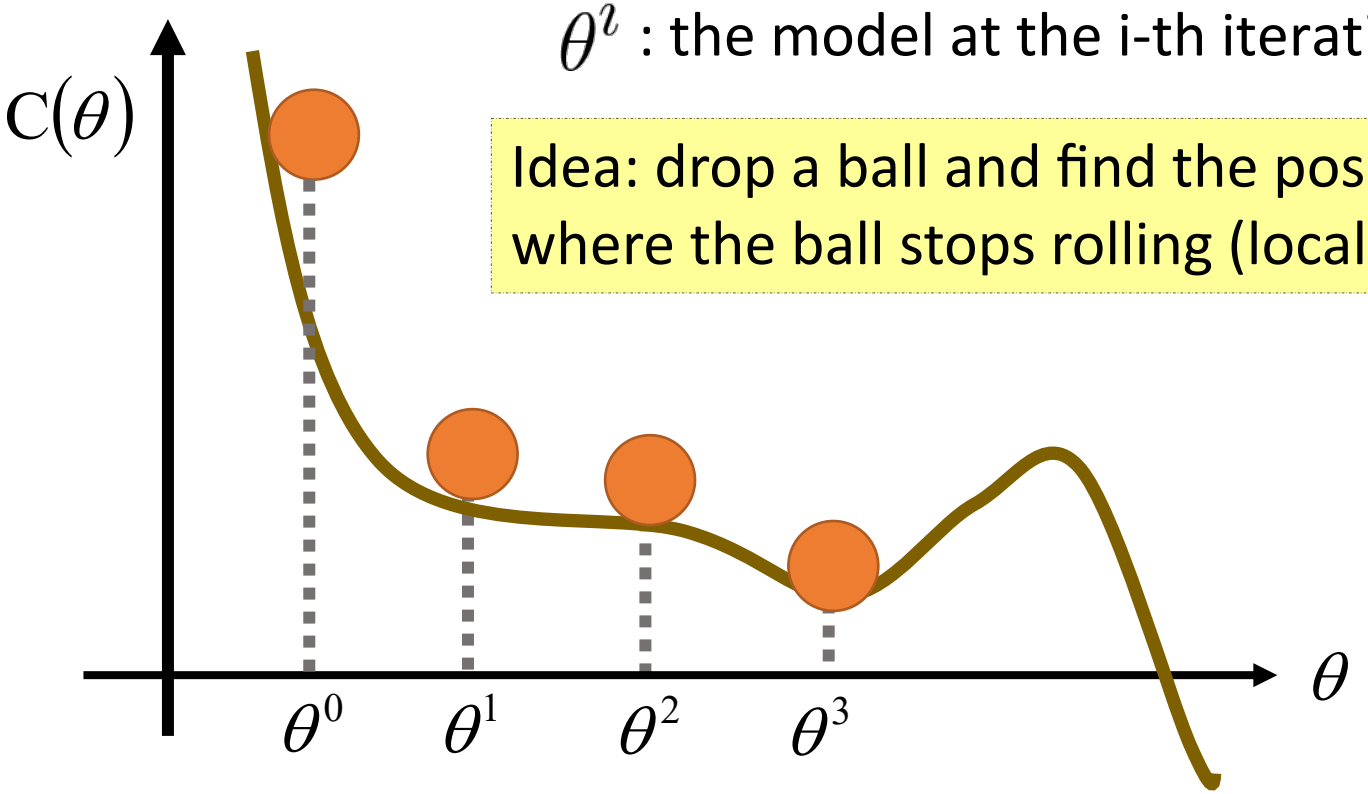
$\mathrm{C}(\theta)$: L ($\mathbf{w}$,b)

Gradient Descent (cont.)

- Assume that θ has only one variable

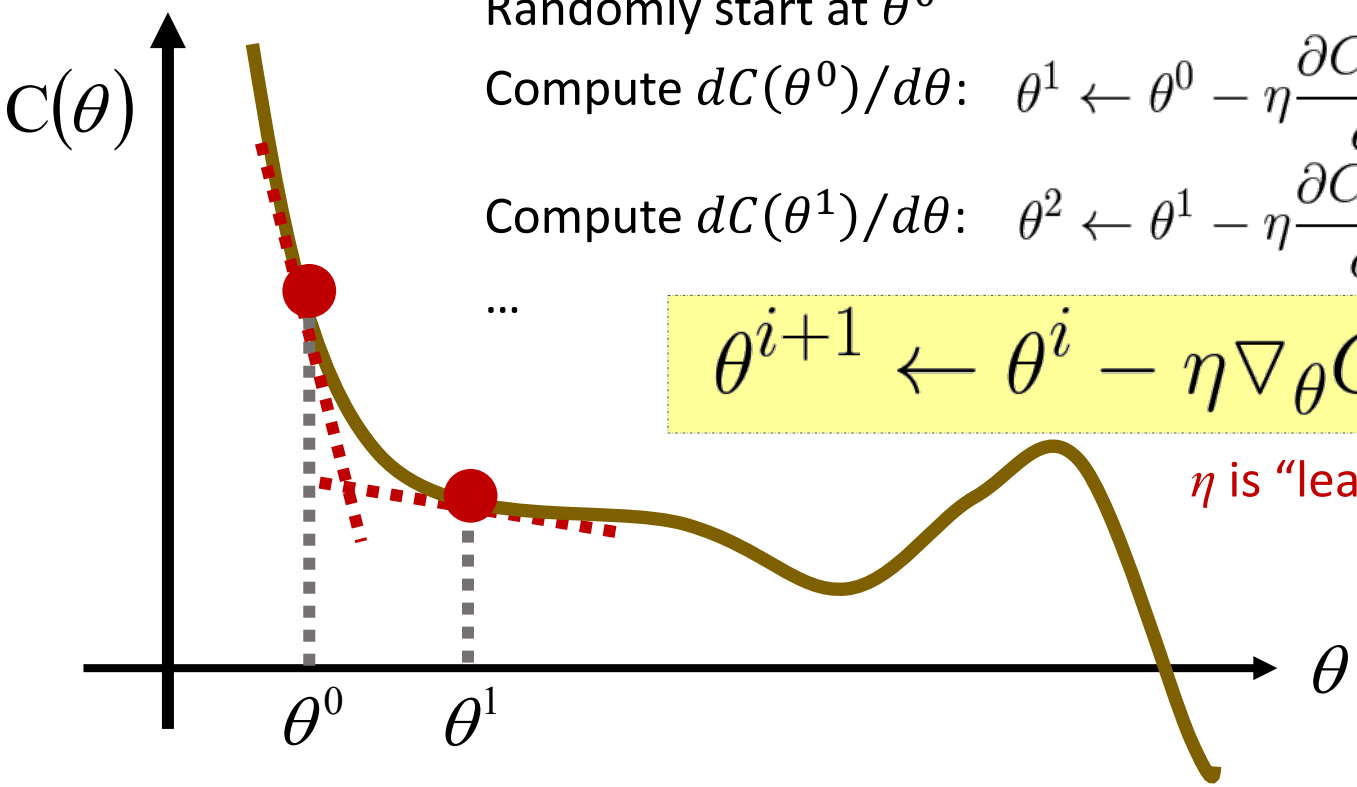
θ^i : the model at the i-th iteration

Idea: drop a ball and find the position where the ball stops rolling (local minima)



Gradient Descent (cont.)

- Assume that θ has only one variable



Randomly start at θ^0

Compute $dC(\theta^0)/d\theta$: $\theta^1 \leftarrow \theta^0 - \eta \frac{\partial C(\theta^0)}{\partial \theta}$

Compute $dC(\theta^1)/d\theta$: $\theta^2 \leftarrow \theta^1 - \eta \frac{\partial C(\theta^1)}{\partial \theta}$

…

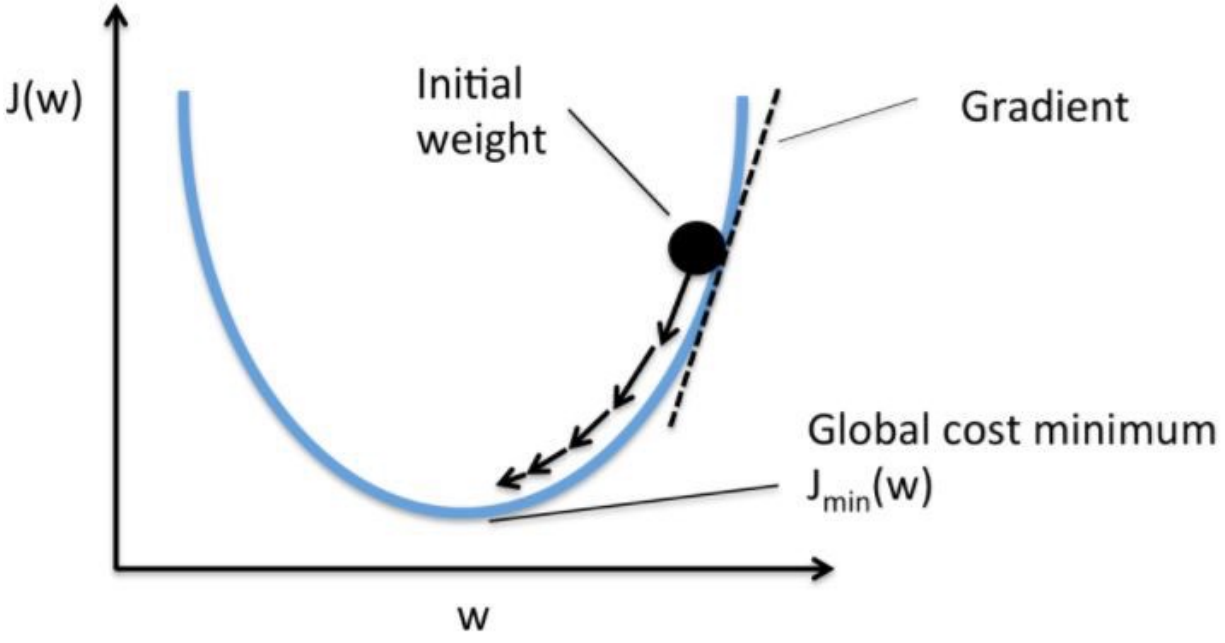
$$\theta^{i+1} \leftarrow \theta^i - \eta \nabla_\theta C(\theta^i)$$

η is "learning rate"

Gradient Descent (cont.)

Base algorithm used to minimize the error with respect to weights. The learning rate determines the step size of the update used to reach to the minimum.

Epoch: One complete pass through all the samples.



Gradient Descent (cont.)

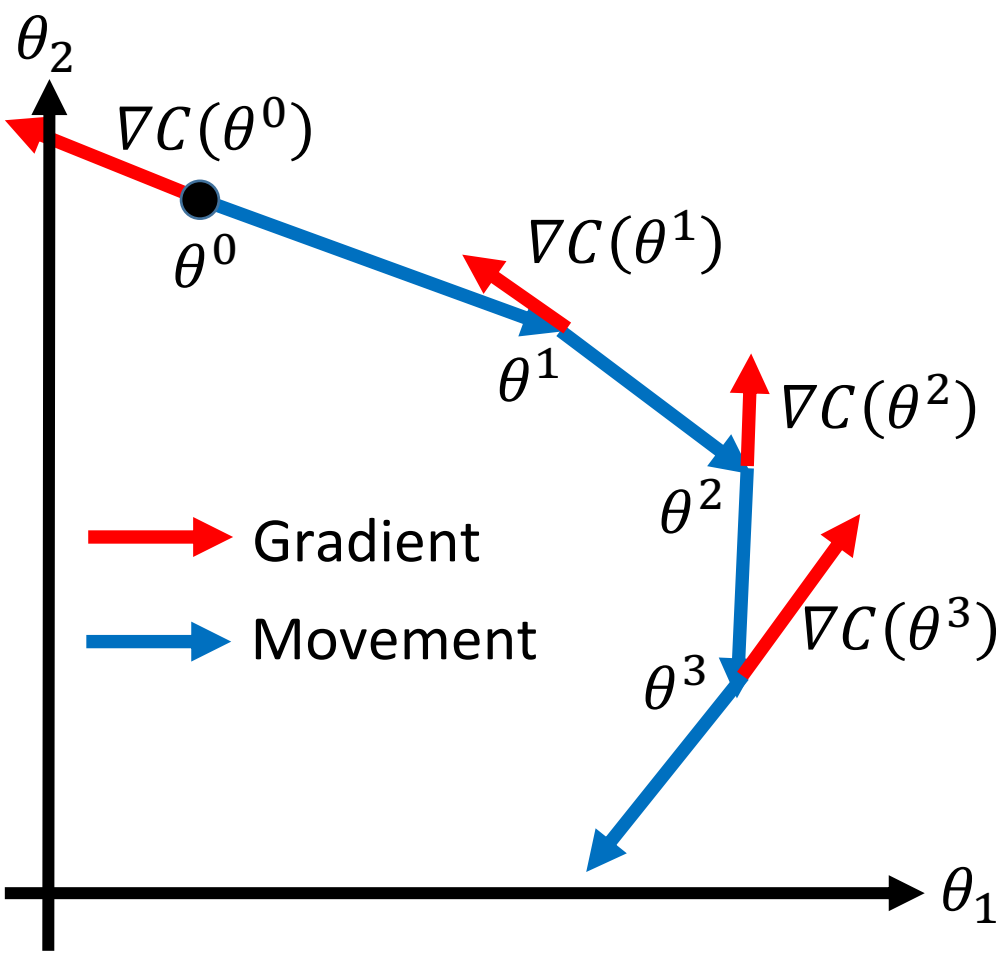
- Assume that θ has two variables $\{\theta_1, \theta_2\}$
  - Randomly start at θ^0: $\theta^0 = \begin{bmatrix} \theta_1^0 \\ \theta_2^0 \end{bmatrix}$
  - Compute the gradients of $C(\theta)$ at θ^0: $\nabla_\theta C(\theta^0) = \begin{bmatrix} \frac{\partial C(\theta_1^0)}{\partial \theta_1} \\ \frac{\partial C(\theta_2^0)}{\partial \theta_2} \end{bmatrix}$
  - Update parameters:

$$\begin{bmatrix} \theta_1^1 \\ \theta_2^1 \end{bmatrix} = \begin{bmatrix} \theta_1^0 \\ \theta_2^0 \end{bmatrix} - \eta \begin{bmatrix} \frac{\partial C(\theta_1^0)}{\partial \theta_1} \\ \frac{\partial C(\theta_2^0)}{\partial \theta_2} \end{bmatrix}$$

$$\theta^{i+1} \leftarrow \theta^i - \eta \nabla_\theta C(\theta^i)$$

  - Compute the gradients of $C(\theta)$ at θ^1: $\nabla_\theta C(\theta^1) = \begin{bmatrix} \frac{\partial C(\theta_1^1)}{\partial \theta_1} \\ \frac{\partial C(\theta_2^1)}{\partial \theta_2} \end{bmatrix}$

Gradient Descent (cont.)



Algorithm

Initialization: start at θ^0

while($\theta^{(i+1)} \neq \theta^i$)

{

compute gradient at θ^i

update parameters

$$\theta^{i+1} \leftarrow \theta^i - \eta \nabla_\theta C(\theta^i)$$

}

An Issue with Gradient Descent

$$\theta^{i+1} = \theta^{i} - \eta \nabla C(\theta^{i})$$

Training Data
$\{(x_1, \hat{y}_1), (x_2, \hat{y}_2), \ldots\}$

$$C(\theta) = \frac{1}{K} \sum_{k} \| f(x_k; \theta) - \hat{y}_k \| = \frac{1}{K} \sum_{k} C_k(\theta)$$

$$\nabla C(\theta^{i}) = \frac{1}{K} \sum_{k} \nabla C_k(\theta^{i})$$

After seeing all training samples, the model can be updated → slow

Stochastic Gradient Descent (SGD)

- Gradient Descent

$$\theta^{i+1} = \theta^i - \eta \nabla C(\theta^i) \quad \nabla C(\theta^i) = \boxed{\frac{1}{K}\sum_k \nabla C_k(\theta^i)}$$

- Stochastic Gradient Descent (SGD)
  - Pick a training sample x_k

$$\theta^{i+1} = \theta^i - \eta \nabla C_k(\theta^i)$$

Training Data
$\{(x_1, \hat{y}_1), (x_2, \hat{y}_2), \ldots\}$

  - If all training samples have the same probability to be picked, then

$$E[\nabla C_k(\theta^i)] = \boxed{\frac{1}{K}\sum_k \nabla C_k(\theta^i)}$$

The model can be updated after seeing one training sample → faster

Epoch (Definition)

- When running SGD, the model starts θ^0

Training Data
$\{(x_1, \hat{y}_1), (x_2, \hat{y}_2), \ldots\}$

pick x_1 $\quad \theta^1 = \theta^0 - \eta \nabla C_1(\theta^0)$

pick x_2 $\quad \theta^2 = \theta^1 - \eta \nabla C_2(\theta^1)$

$\vdots$

pick x_k $\quad \theta^k = \theta^{k-1} - \eta \nabla C_k(\theta^{k-1})$

$\vdots$

pick x_K $\quad \theta^K = \theta^{K-1} - \eta \nabla C_K(\theta^{K-1})$

see all training samples once

→ one epoch

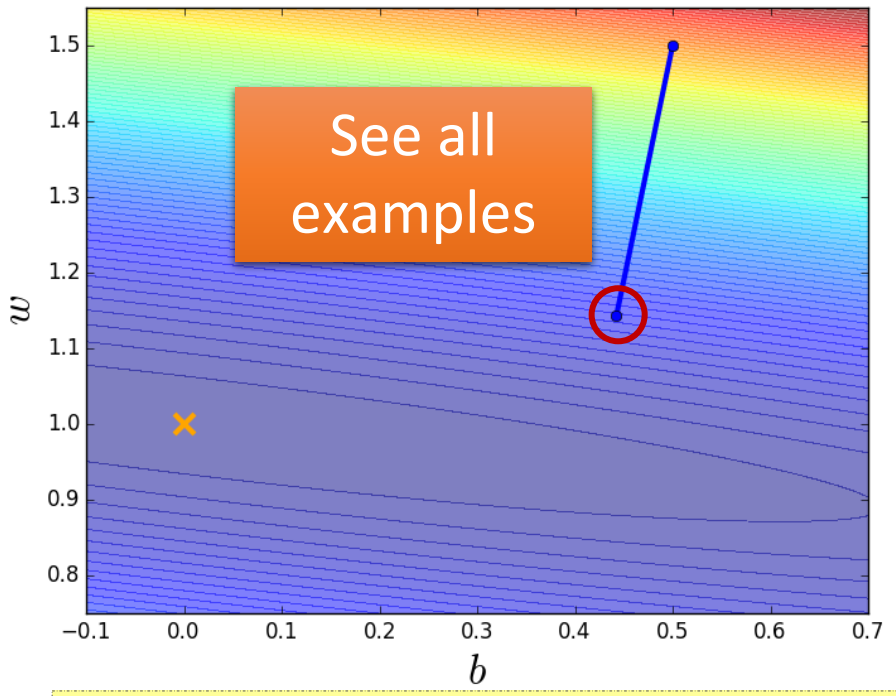
---

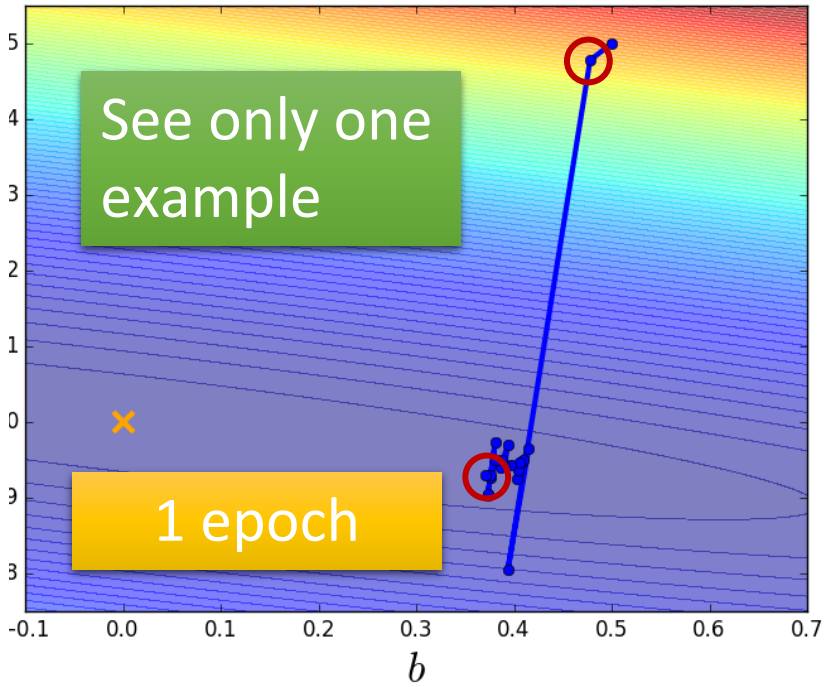
pick x_1 $\quad \theta^{K+1} = \theta^K - \eta \nabla C_1(\theta^K)$

Gradient Descent v.s. SGD

- Gradient Descent
- Update after seeing all the examples

- Stochastic Gradient Descent
- If there are 20 examples, update 20 times in one epoch.





SGD approaches the target faster than gradient descent

Mini-Batch SGD

- Batch Gradient Descent

$$\theta^{i+1} = \theta^i - \eta \frac{1}{K} \sum_k \nabla C_k(\theta^i)$$

Use all K samples in each iteration

- Stochastic Gradient Descent (SGD)
  - Pick a training sample x_k

$$\theta^{i+1} = \theta^i - \eta \nabla C_k(\theta^i)$$

Use 1 samples in each iteration

- Mini-Batch SGD
  - Pick a set of B training samples as a batch b

B is "batch size"

$$\theta^{i+1} = \theta^i - \eta \frac{1}{B} \sum_{x_k \in b} \nabla C_k(\theta^i)$$

Use all B samples in each iteration

Mini-Batch SGD

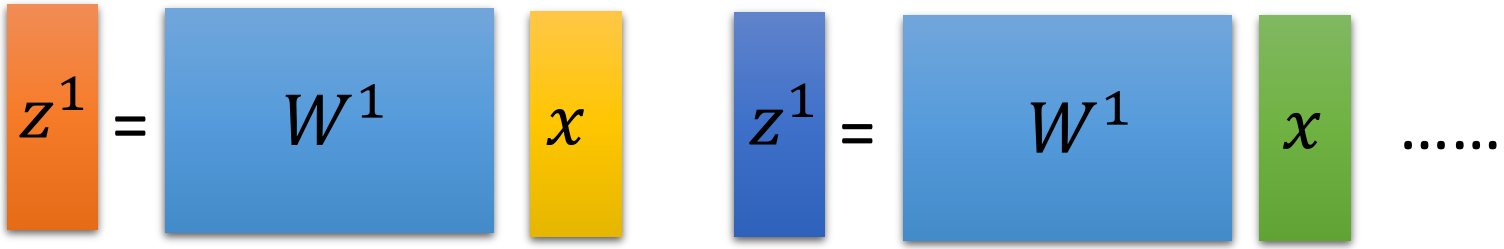
Algorithm 8.1 Stochastic gradient descent (SGD) update at training iteration k

Require: Learning rate ϵ_k.
Require: Initial parameter $\boldsymbol{\theta}$
  while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient estimate: $\hat{\boldsymbol{g}} \leftarrow +\frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \hat{\boldsymbol{g}}$
  end while

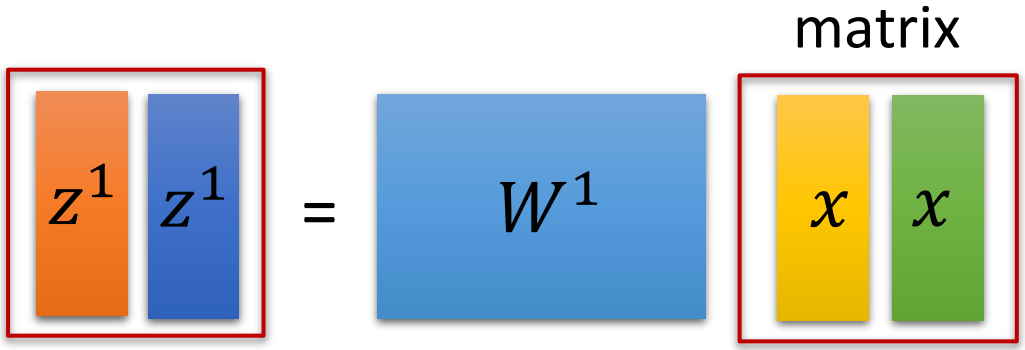
SGD vs. Mini-Batch

Training speed: mini-batch > SGD > Gradient Descent

- Stochastic Gradient Descent (SGD)



- Mini-Batch SGD



Modern computers run matrix-matrix multiplication faster than matrix-vector multiplication

Topics for Next Week

- Linear Neural Networks for Regression
- Practical Tips
- Linear Neural Networks for Classification
- Activation and Loss Functions Using PyTorch
- Multi-Layer Perceptron
- Homework1 assigned